

PROGRAMMATION FONCTIONNELLE 2 : TD7 (ARBRES, ENTIERS EN BINAIRE)

mars 2026 — Pierre Rousselin

Exercice 1 : arbres binaires

On rappelle la définition des arbres binaires (non stricts) donnée en cours :

```
Inductive btree (A : Type) :=
| emptyTree
| node (left : btree A) (key : A) (right : btree A).
```

1. Définir les fonctions `height` (hauteur d'un arbre, la hauteur de l'arbre vide est 0) et `numNodes` (nombre de sommets).
2. Définir l'opération `mirror` qui change gauche en droite dans tout l'arbre.
3. Donner la preuve en Rocq, puis la « preuve papier très détaillée » du lemme suivant :
`Lemma numNodes_mirror {A : Type} (t : btree A) : numNodes (mirror t) = numNodes t.`
 De quels résultats supplémentaires avez-vous eu besoin ?
4. Définir la fonction `prefixDFS : btree A -> list A` qui donne la liste de toutes les clés dans l'arbre en commençant par la racine, puis en parcourant le sous-arbre gauche et enfin le sous-arbre droit.
5. Que dire, pour un arbre binaire `t` de `length (prefixDFS t)` ? Le prouver avec une « preuve papier très détaillée » et/ou une preuve en Rocq. De quels résultats supplémentaires avez-vous eu besoin ?
6. Un arbre binaire parfait est un arbre dont tous les niveaux sont pleins et toutes les feuilles sont à la même hauteur. Définir la fonction `perfect : nat -> A -> btree A` qui construit un tel arbre, dont la hauteur est le premier argument et dont toutes les clés sont égales à son deuxième argument.
7. Prouver que `perfect n` a bien hauteur `n`.
8. Définir la puissance d'un entier par un autre entier, puis donner la preuve papier très détaillée de :
`Lemma numNodes_perfect {A : Type} (n : nat) (a : A) :`
`S (numNodes (perfect n a)) = 2 ^ n.`
 où l'opérateur `^` désigne la puissance.

--- * ---

Exercice 2 : Arbres binaires stricts

On rappelle la définition des arbres binaires stricts :

```
Inductive sbtree (A : Type) :=
| leaf (key : A)
| node (left : sbtree A) (key : A) (right : sbtree A).
```

1. Rappeler la définition des fonctions `numInnerNodes` (nombre de nœuds internes) et `numLeaves` (nombre de feuilles).
2. Quelle relation y a-t-il entre ces deux valeurs ? Énoncer un lemme en Rocq et en donner la preuve.
3. Cette relation est-elle encore valable pour des arbres binaires non stricts ?
4. Écrire une fonction qui permet d'encoder un `btree A` dans un `sbtree (option A)`, où `option` est le type
`Inductive option {A : Type} := Some (a : A) | None.`
 Cette fonction est-elle bijective ?

--- * ---